

Evidence review

The harness is part of the evaluated system

Public studies show that the coding-agent harness changes performance, cost, token use and runtime. They do not identify one harness that wins across every model and task.

Evidence captured on 12 July 2026

13 benchmarks, ordered by how much they can tell us

The evidence is consistent on one point: changing the harness can change quality, cost and speed. It is not consistent on which harness wins.

Harnesses are not neutral
Tools, context, retries and timeouts change the evaluated system.

The ranking changes
No harness wins across every model and task.

Test the combination
Evaluate model, harness, task and budget together.

1 Operational and broad evidence

Start here: larger, more operational or more extensively reported comparisons.

01	Databricks coding-agent benchmark
02	Warden benchmark
03	Coding Agent Index
04	DrugDiscoveryBench
05	Agent Security League

2 Focused domain evidence

Strong task-specific evidence, but conclusions may not transfer outside the domain.

06	scBench-Long
07	Vulnerability-Agent-Bench
08	WildClawBench
09	CORE-Bench Hard

3 Small and diagnostic evidence

Useful signals with limited task counts or no direct measure of coding quality.

10	HarnessBench
11	A-CODE-CLI
12	OpenBench M4.5
13	Harness Tax

Databricks evaluated recent internal pull requests: Pi used about one-third as much context, and equal-quality runs sometimes differed by more than 2 times in cost.

What they did

Databricks replayed recent human-written pull requests from its multi-million-line internal codebase. Engineers reviewed whether each agent's changes passed repository tests and matched the intended change.

Tasks	Not published
Repeated trials	not clear
Grader	Repository tests reviewed by engineers

What we observe

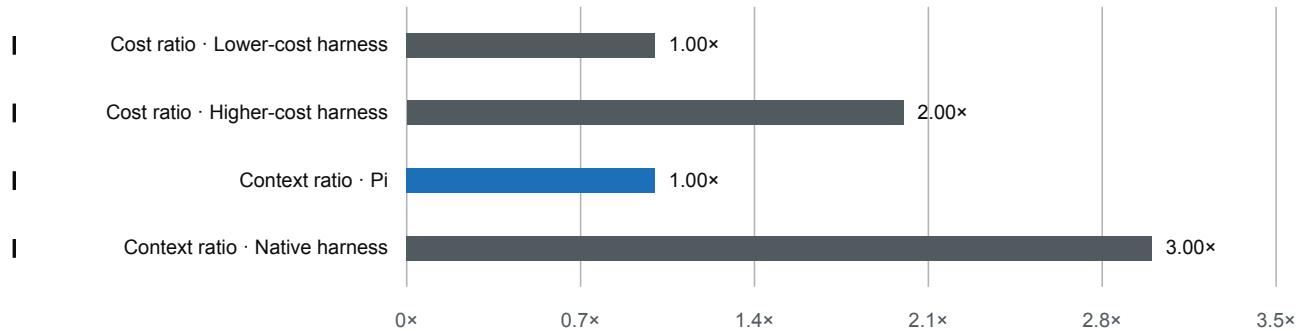
Pi used about one-third as much context per turn. Some equal-quality matched runs differed by more than 2 times in cost.

Limit

The source does not publish task count, matched rows, run-level data or uncertainty.

Databricks results

Published results



The source reports 'more than 2 times' for cost and 'about 3 times' for context. These are directional ratios, not exact run-level values.

Sentry tested 86 known vulnerabilities: Pi found a similar or larger share while costing about one-quarter to one-fifth as much as Claude SDK.

What they did

Sentry gave agents files connected to 86 historical vulnerabilities and measured how many known findings they recovered. The published table compares matched Claude models running in Pi and Claude SDK.

Tasks	86
Repeated trials	no
Grader	Known-corpus recall

What we observe

Pi cost less and found a similar or larger share of the known corpus for the matched Claude models.

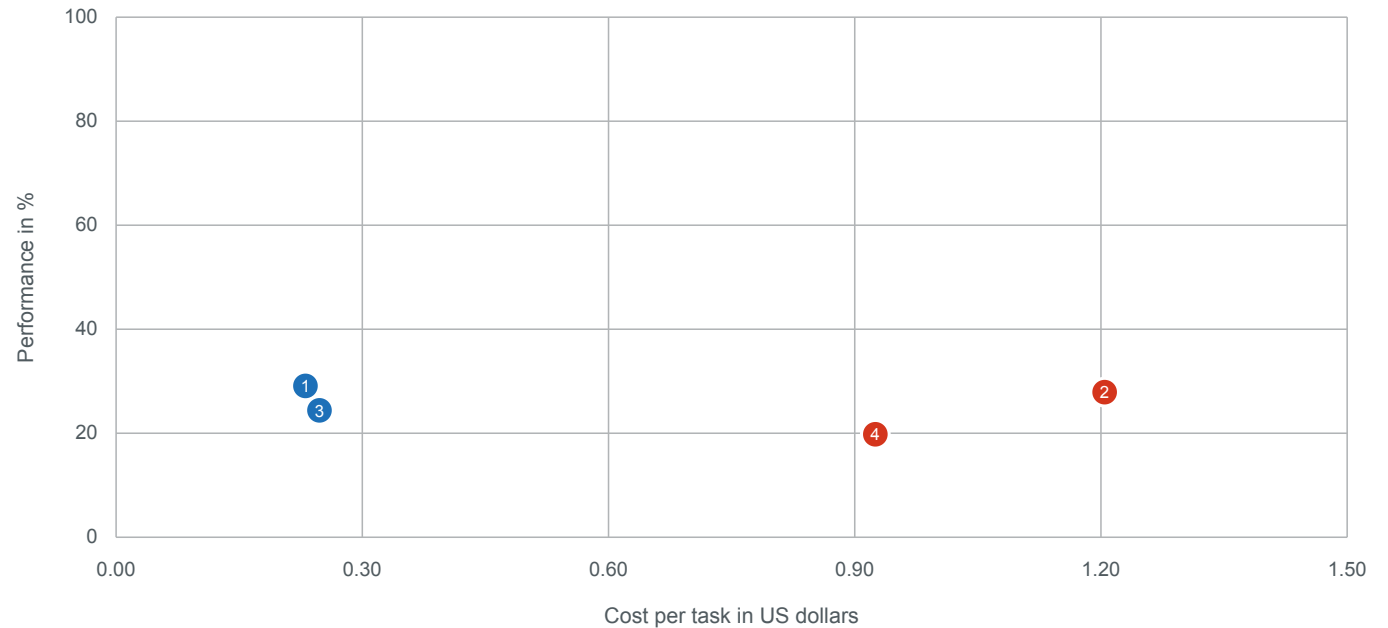
Limit

The comparator is Claude SDK, not the Claude Code command-line product. The benchmark does not measure whole-repository discovery.

Sentry results

Dataset or repository

Published results



Point	Model	Harness	Result · cost
1	Claude Sonnet 4.6	Pi	29.1% · \$0.23
2	Claude Sonnet 4.6	Claude SDK	27.9% · \$1.20
3	Claude Opus 4.8	Pi	24.4% · \$0.25
4	Claude Opus 4.8	Claude SDK	19.8% · \$0.93

Artificial Analysis combined 321 coding tasks: OpenCode scored higher with Opus 4.7, while Claude Code was cheaper and faster.

What they did

Artificial Analysis combines 321 tasks from DeepSWE, Terminal-Bench v2 and SWE-Atlas-QnA. It repeats each configuration and reports a composite score alongside cost and runtime.

Tasks	321
-------	-----

Repeated trials	yes
-----------------	-----

Grader	Program checks, test suites and rubric scoring
--------	--

What we observe

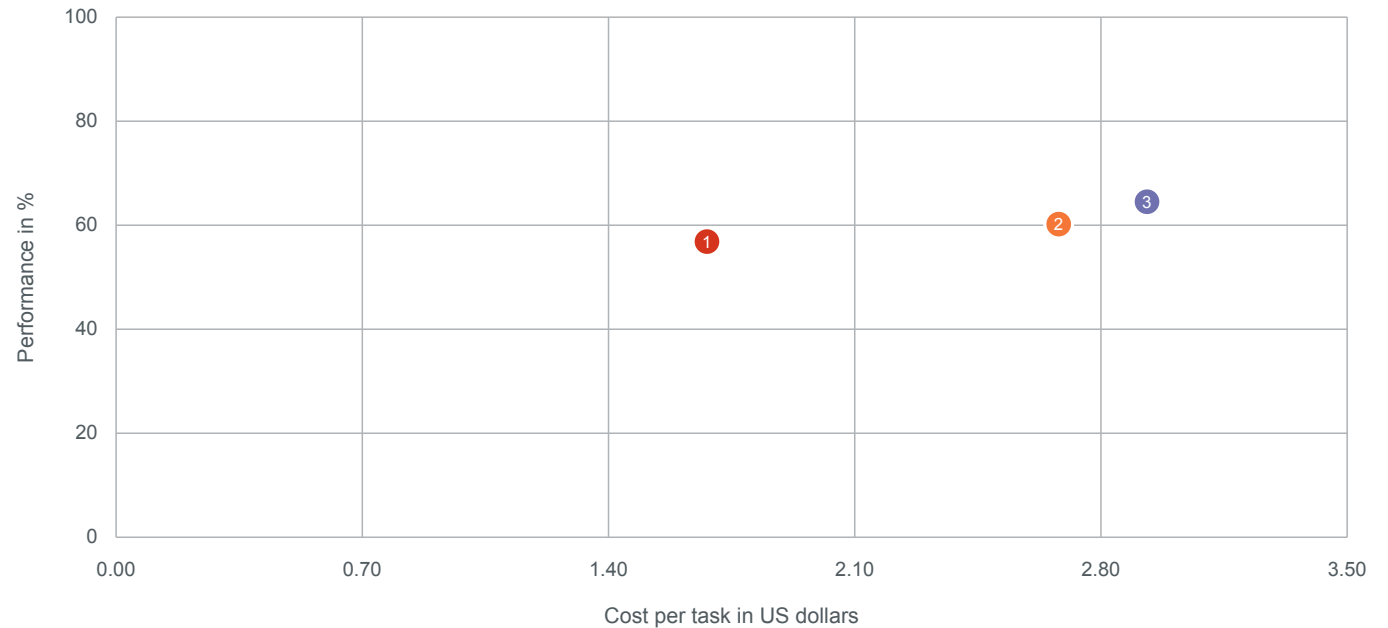
OpenCode led the captured Opus 4.7 medium comparison. Claude Code was cheaper and faster.

Limit

The index averages 3 different benchmark families. The live results and harness versions can change.

Artificial Analysis results

Published results



Point	Model	Harness	Result · cost
1	Claude Opus 4.7	Claude Code	56.9% · \$1.68
2	Claude Opus 4.7	Cursor CLI	60.2% · \$2.68
3	Claude Opus 4.7	OpenCode	64.5% · \$2.93

DrugDiscoveryBench repeated 82 expert workflows: mini-SWE-agent led the matched GPT-5.5 comparison, while Codex placed in the middle.

What they did

Scale AI ran 82 expert-curated drug-discovery and life-sciences workflows 3 times. An LLM judge scored each result against expert-authored rubrics and reference answers.

Tasks	82
Repeated trials	yes
Grader	Expert-authored rubrics scored by an LLM judge

What we observe

GPT-5.5 performed best in mini-SWE-agent. Codex was neither the best nor worst harness for that model.

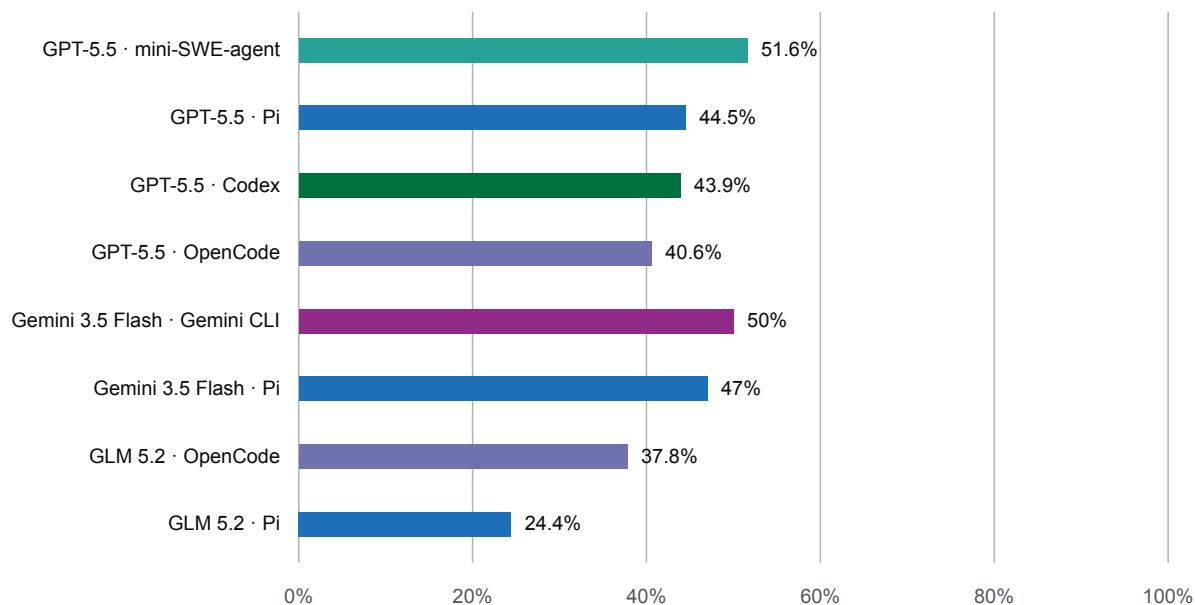
Limit

The paper plots cost but does not publish a table of exact cost points. The outcome depends on an LLM judge.

Scale AI results

Dataset or repository

Published results



Endor Labs tested 200 vulnerability repairs: Cursor led for Opus, but Claude Code led for Sonnet 5.

What they did

Endor Labs asked each configuration to repair 200 real vulnerabilities. Hidden tests separately measured whether the software still worked and whether the vulnerability was fixed.

Tasks	200
Repeated trials	no
Grader	Hidden functional and security tests

What we observe

Cursor led Claude Code for Opus 4.7 and 4.8. Claude Code led Cursor for Sonnet 5.

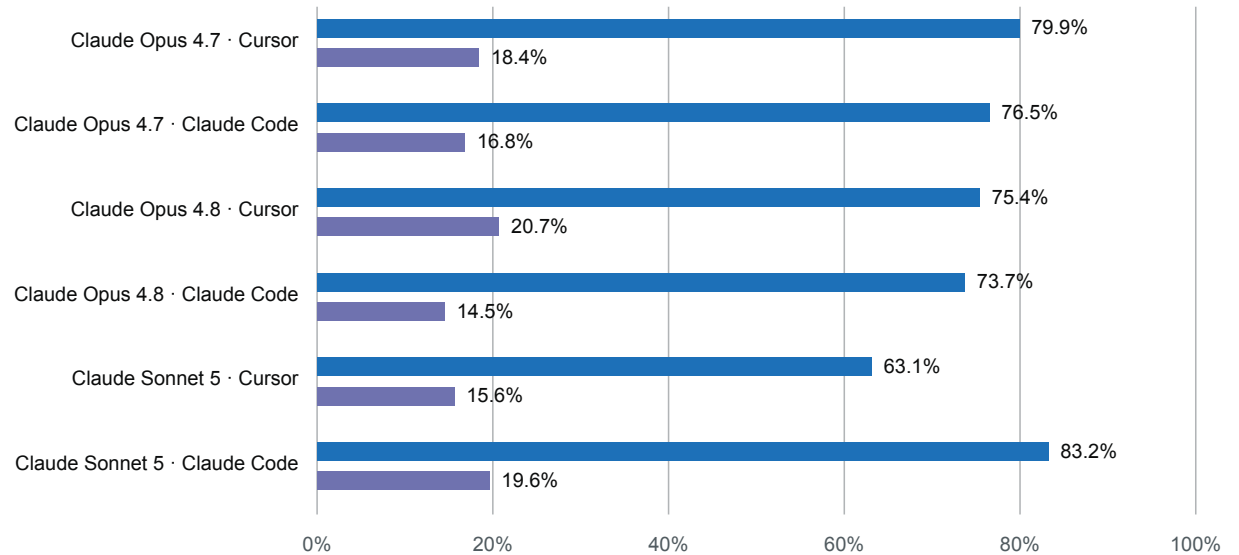
Limit

There are no repeated trials or task-level costs. Timeouts explain part of the reversal.

Endor Labs results

Published results

Functional pass rate Security pass rate



scBench-Long repeated 21 biological workflows: the apparent harness differences are smaller than the reported uncertainty.

What they did

LatchBio ran 21 long biological analysis workflows 3 times for each model and harness combination. Deterministic biological graders checked whether the analyses reached the required outputs.

Tasks	21
Repeated trials	yes
Grader	Deterministic biological graders

What we observe

Claude Code led Pi on Opus 4.8 while Codex and Pi tied on GPT-5.5. The reported intervals overlap widely.

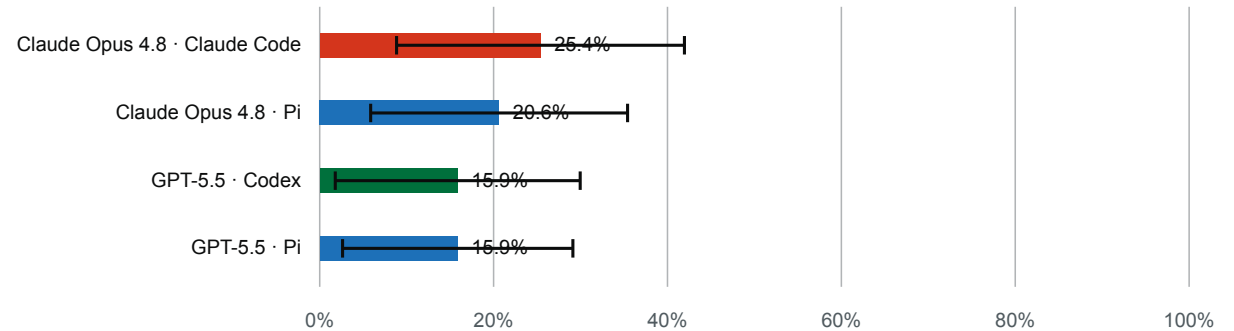
Limit

The public summary does not expose cost. The confidence intervals are wide.

LatchBio results

Dataset or repository

Published results



XOR tested 136 vulnerability repairs: Claude Code and Codex beat OpenCode on all 3 published matched-model pairs.

What they did

XOR ran agents in isolated containers on real vulnerability-repair tasks. Its public explorer reports pass rate and cost for matched model and harness configurations.

Tasks	136
Repeated trials	no
Grader	Vulnerability repair checks

What we observe

Claude Code and Codex led the corresponding OpenCode configurations on the published matched pairs.

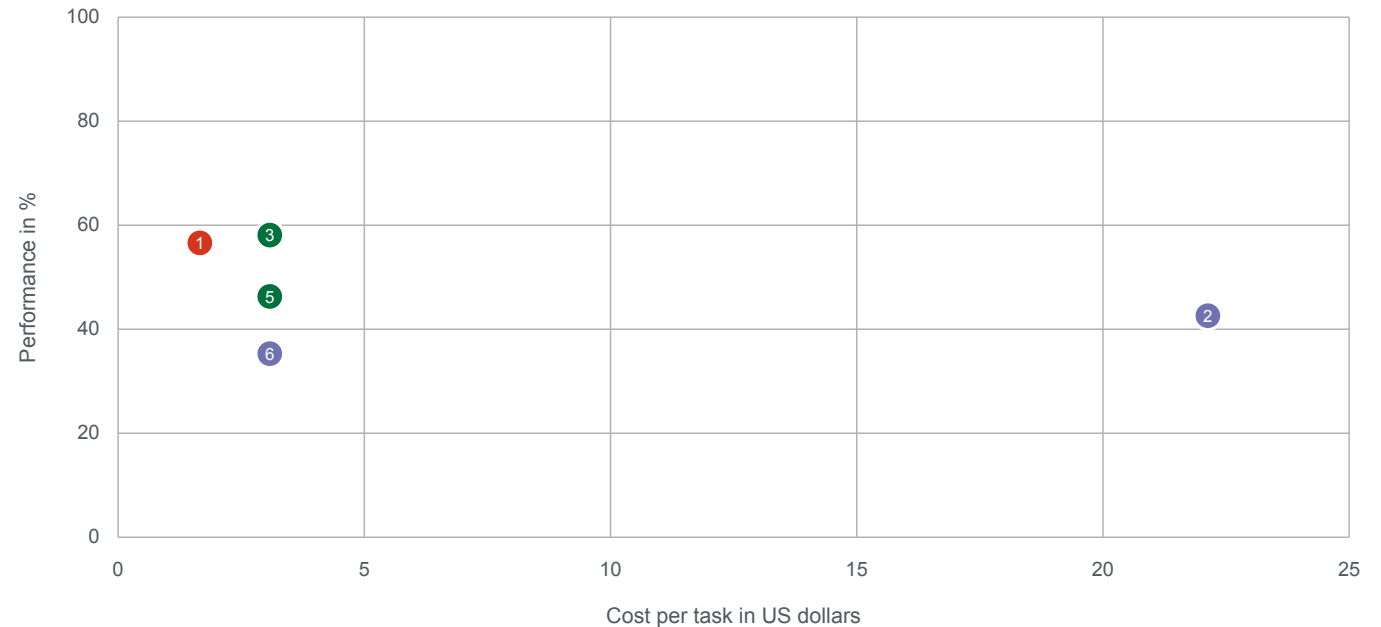
Limit

The site alternates between 128 and 136 samples. Some reported cost differences need more explanation.

XOR results

Dataset or repository

Published results



Point	Model	Harness	Result · cost
1	Claude Opus 4.6	Claude Code	56.6% · \$1.66
2	Claude Opus 4.6	OpenCode	42.6% · \$22.13
3	GPT-5.2	Codex	58.1% · \$3.08
4	GPT-5.2	OpenCode	46.3% · \$3.08
5	GPT-5.2-Codex	Codex	46.3% · \$3.08
6	GPT-5.2-Codex	OpenCode	35.3% · \$3.08

WildClawBench tested 60 long-horizon tasks: changing the harness produced double-digit score shifts and changed the winner by model.

What they did

InternLM evaluated 60 bilingual, multimodal and long-horizon tasks in harness-specific container images. Deterministic checks, environment checks and model judges contributed to the overall score.

Tasks	60
Repeated trials	not clear
Grader	Deterministic checks, environment checks and model judges

What we observe

The best harness changed with the model. The source reports double-digit score changes from switching harness.

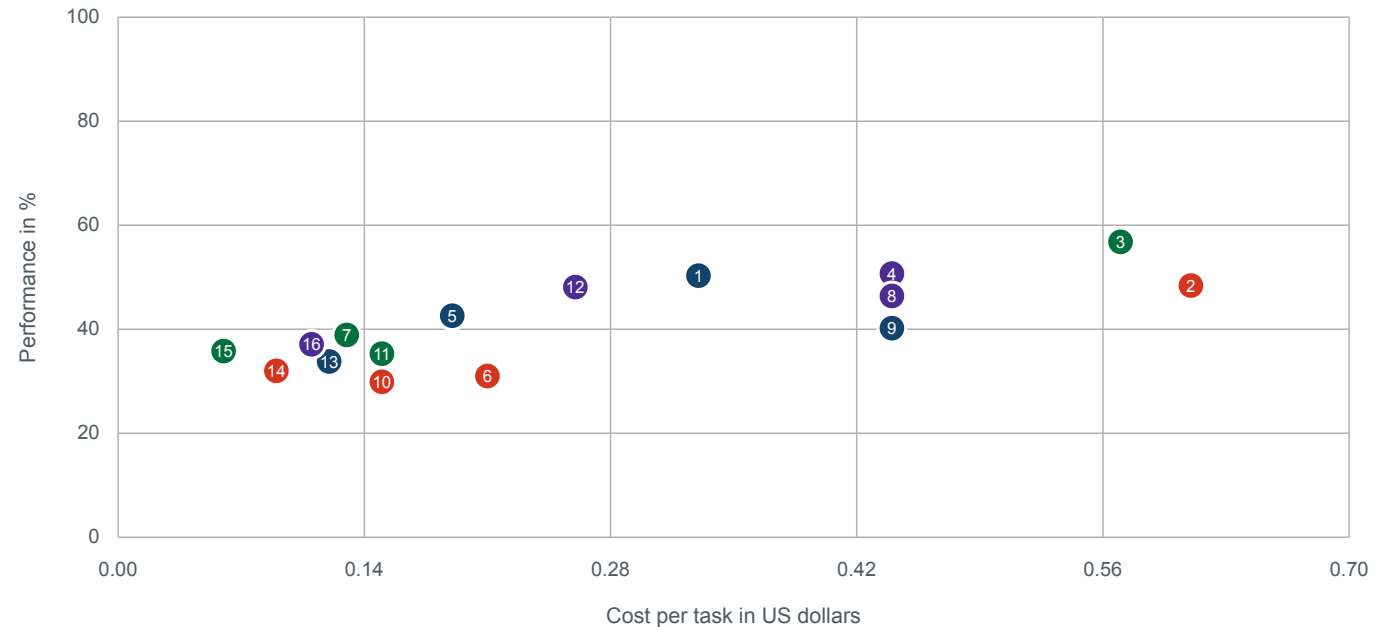
Limit

Each harness image has different tools and prompt conventions. Repeat counts are not clear in the headline table.

InternLM results

Dataset or repository

Published results



Point	Model	Harness	Result · cost	Point	Model	Harness	Result · cost
1	GPT-5.4	OpenClaw	50.3% · \$0.33	9	MiMo V2 Pro	OpenClaw	40.2% · \$0.44
2	GPT-5.4	Claude Code	48.4% · \$0.61	10	MiMo V2 Pro	Claude Code	29.9% · \$0.15
3	GPT-5.4	Codex	56.8% · \$0.57	11	MiMo V2 Pro	Codex	35.3% · \$0.15
4	GPT-5.4	Hermes Agent	50.7% · \$0.44	12	MiMo V2 Pro	Hermes Agent	48.1% · \$0.26
5	GLM 5	OpenClaw	42.6% · \$0.19	13	MiniMax M2.7	OpenClaw	33.8% · \$0.12
6	GLM 5	Claude Code	31% · \$0.21	14	MiniMax M2.7	Claude Code	32% · \$0.09
7	GLM 5	Codex	38.9% · \$0.13	15	MiniMax M2.7	Codex	35.8% · \$0.06
8	GLM 5	Hermes Agent	46.4% · \$0.44	16	MiniMax M2.7	Hermes Agent	37.1% · \$0.11

CORE-Bench Hard tested 45 scientific reproductions: Claude Code more than doubled verified success while costing less than CORE-Agent.

What they did

Princeton HAL asked agents to reproduce 45 research artefacts. The leaderboard verifies whether each system recreated the required scientific result and reports total cost.

Tasks	45
-------	----

Repeated trials	no
-----------------	----

Grader	Research artifact reproduction
--------	--------------------------------

What we observe

Claude Code more than doubled the verified success of CORE-Agent for Claude Opus 4.5 while costing less.

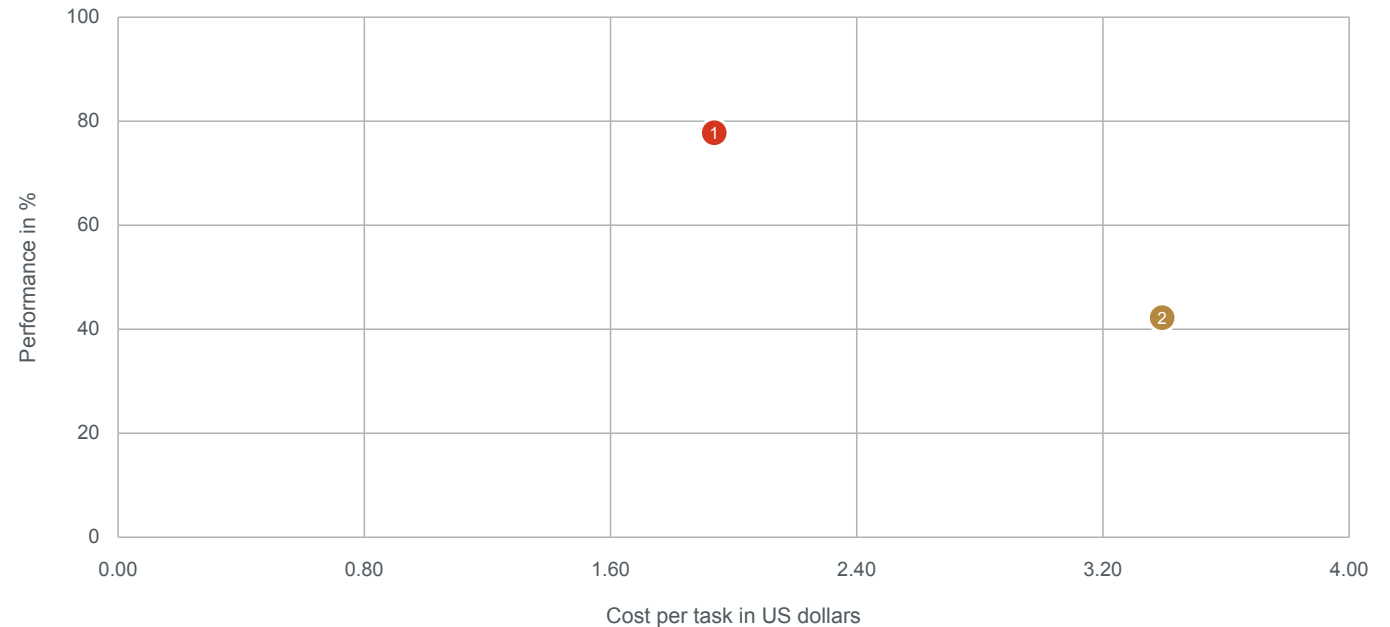
Limit

There is one run per configuration. The systems differ across retrieval, tools and context policy.

Princeton HAL results

Dataset or repository

Published results



Point	Model	Harness	Result · cost
1	Claude Opus 4.5	Claude Code	77.8% · \$1.94
2	Claude Opus 4.5	CORE-Agent	42.2% · \$3.39

HarnessBench tested 27 recent bug fixes: several configurations finished within one task of the leader, but runtime differed more clearly.

What they did

HarnessBench selected 27 recent bug-fix issues from 9 open-source repositories. Hidden tests checked the core fix and regressions, with one run for each configuration.

Tasks	27
Repeated trials	no
Grader	Hidden core and regression tests

What we observe

Several conditions finished within one task of the observed leader. Runtime differences were clearer than pass-rate differences.

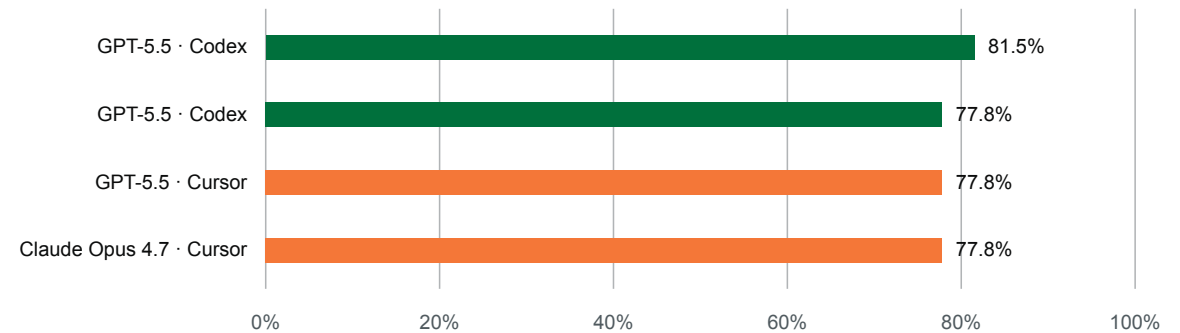
Limit

The author states that the 27-task pass-rate differences are not statistically significant.

Nyosegawa results

Dataset or repository

Published results



A-CODE-CLI repeated 10 web scenarios: OpenCode had a 2.4-point backend lead and cost 44% less than Claude Code.

What they did

AIMultiple ran 10 full-stack web development scenarios 3 times with Claude Sonnet 4.6. Backend checks and Playwright tests measured correctness and user-interface behaviour.

Tasks	10
Repeated trials	yes
Grader	Backend checks and Playwright user-interface checks

What we observe

OpenCode had a small backend lead over Claude Code and cost less in the matched Sonnet 4.6 comparison.

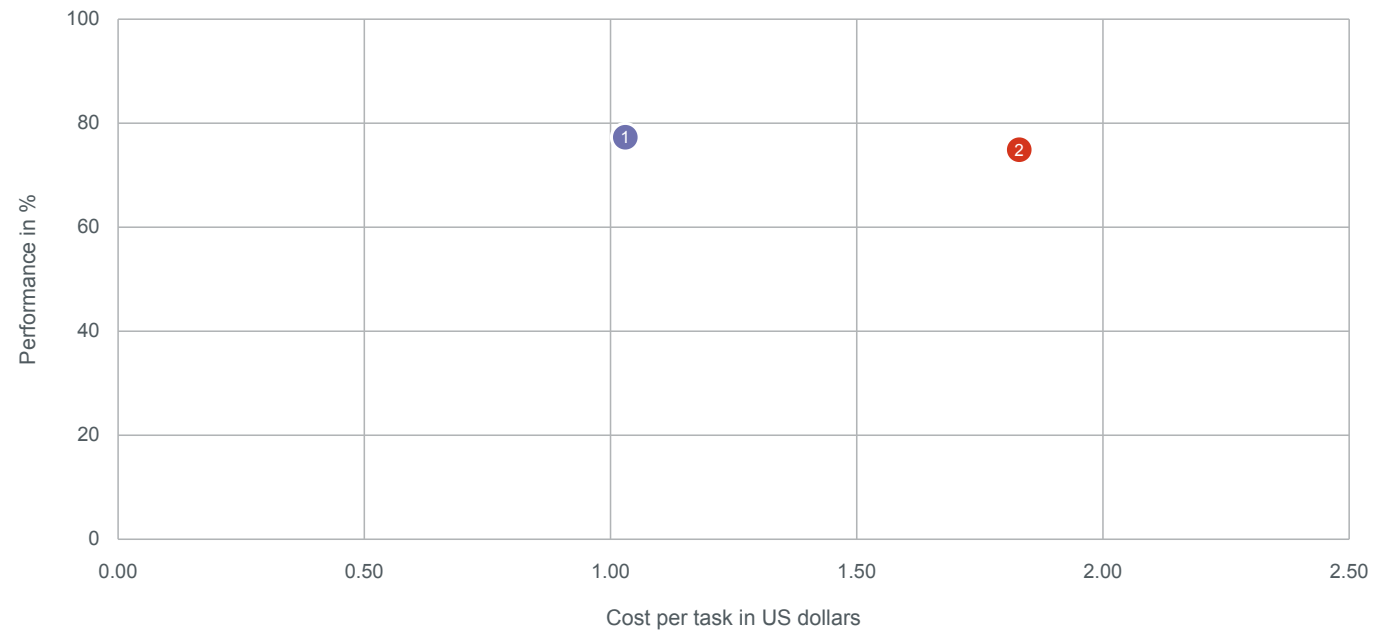
Limit

The benchmark has only 10 scenarios. Only one example task is public.

AIMultiple results

Dataset or repository

Published results



Point	Model	Harness	Result · cost
1	Claude Sonnet 4.6	OpenCode	77.3% · \$1.03
2	Claude Sonnet 4.6	Claude Code	74.9% · \$1.83

OpenBench repeated 3 coding tasks: every harness passed, so the useful difference was efficiency rather than correctness.

What they did

OpenBench ran 3 repository coding tasks 3 times with GPT-5.5 at medium effort. Executable checkers awarded partial credit, while the study also recorded wall time and token use.

Tasks	3
Repeated trials	yes
Grader	Executable checkers with partial-credit support

What we observe

All clean harnesses passed every task. Pi and Cursor used less time and fewer tokens.

Limit

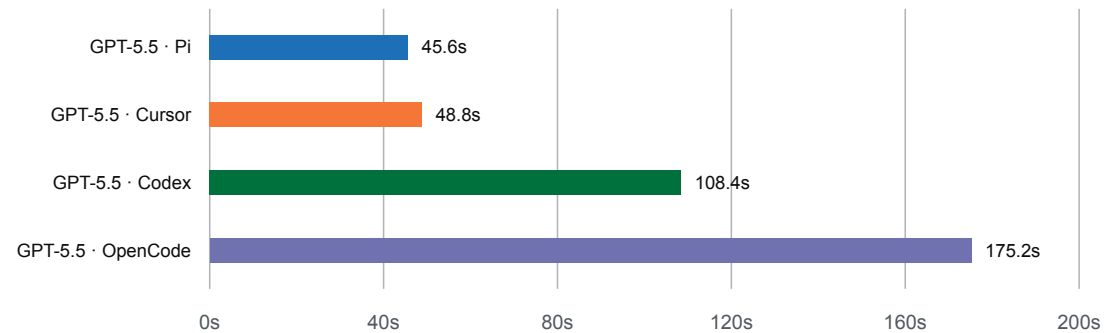
The tasks were too easy to separate frontier harnesses on correctness.

OpenBench results

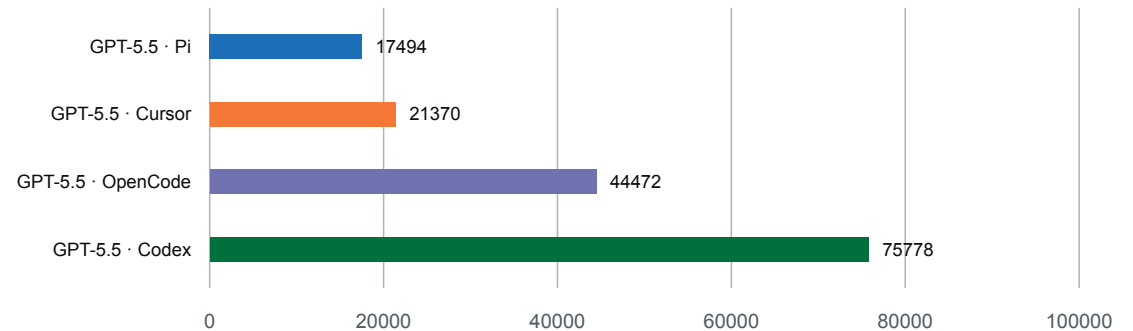
Dataset or repository

Published results

Mean wall time



Tokens per solve



Portkey measured 2 trivial requests: fixed input overhead ranged from about 2,600 tokens for Pi to 27,000 for Claude Code.

What they did

Portkey routed 2 trivial prompts through its gateway and recorded the input payload sent by each harness. The exercise isolates fixed prompt overhead rather than coding performance.

Tasks	2
Repeated trials	no
Grader	Input-token capture

What we observe

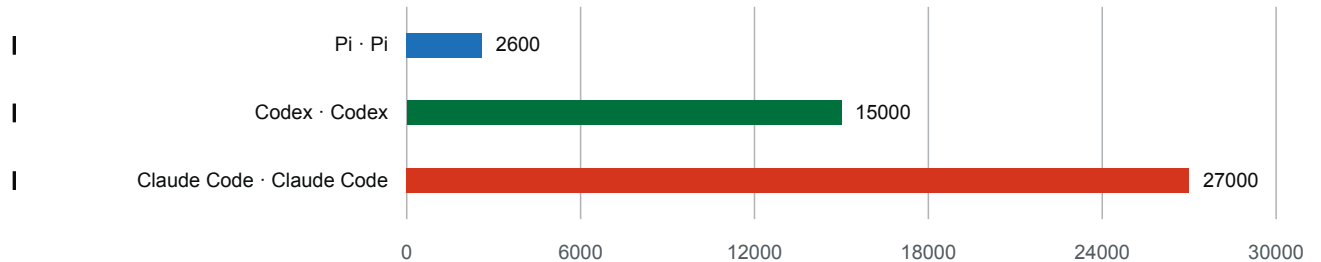
The fixed input payload ranged from about 2,600 tokens for Pi to 27,000 for Claude Code.

Limit

This is an overhead diagnostic. It does not measure coding quality or full-session economics.

Portkey results

Published results



This diagnostic measures fixed input overhead. It does not measure coding quality.

Evidence matrix

Twelve studies contain at least one matched-model harness comparison. Portkey is an efficiency diagnostic, not a quality benchmark.

No.	Study	Task surface	Model fixed	Includes Pi	Repeats	Cost
01	Databricks coding-agent benchmark	Recent human-written pull requests from a multi-million-line internal codebase	yes	yes	not clear	yes
02	Warden benchmark	Known-vulnerability recognition in files tied to historical Sentry vulnerabilities	yes	yes	no	yes
03	Coding Agent Index	321 tasks across DeepSWE, Terminal-Bench v2 and SWE-Atlas-QnA	yes	no	yes	yes
04	DrugDiscoveryBench	Expert-curated drug-discovery and life-sciences workflows	yes	yes	yes	yes
05	Agent Security League	Real vulnerability-fixing tasks with hidden functional and security tests	yes	no	no	no
06	scBench-Long	Long biological analysis workflows	yes	yes	yes	no
07	Vulnerability-Agent-Bench	Real vulnerability repairs in isolated containers	yes	no	no	yes
08	WildClawBench	Bilingual, multimodal and long-horizon agent tasks	yes	no	not clear	yes
09	CORE-Bench Hard	Scientific reproducibility tasks	yes	no	no	yes
10	HarnessBench	Recent bug-fix issues from 9 open-source repositories	yes	no	no	yes
11	A-CODE-CLI	Ten full-stack web development scenarios	yes	no	yes	yes
12	OpenBench M4.5	Three repository coding tasks with executable checkers	yes	yes	yes	no
13	Harness Tax	Two trivial prompts routed through a gateway	no	yes	no	no

[How to read harness comparisons](#) · [Download observations](#) · [Download study records](#)

Data and source access

External datasets remain under their publishers' terms. This repository stores derived observations and links to the original data.

Source	Access	Licence note	Links
Benchmarking coding agents on Databricks' multi-million-line codebase	No public task-level dataset found	Not stated for benchmark data	Results
Warden benchmark	Public aggregate table	Not stated for benchmark results	Results · Data
scBench-Long	Public JSON summary and public trajectories	Check the linked benchmark repository and dataset terms	Results · Data
OpenBench	Public repository with committed data for several milestones	MIT	Results · Data
DrugDiscoveryBench	Tasks and code are public; rubrics and reference answers are gated on Hugging Face	MIT for repository code and tasks; separate terms apply to bundled data and gated material	Results · Data
Artificial Analysis Coding Agent Index	Live public result page and methodology	Not stated for result data	Results
Endor Labs Agent Security League	Public leaderboard and articles	Not stated for benchmark data	Results
Vulnerability-Agent-Bench	Public explorer and linked Weights and Biases project	Not stated for benchmark data	Results · Data
WildClawBench	Public code, tasks, runtime images and dataset	Check the repository and dataset cards	Results · Data
HarnessBench	Public code and results	Check the repository licence	Results · Data
CORE-Bench Hard	Public harness, leaderboard and encrypted traces	Check the repository licence	Results · Data
A-CODE-CLI	Public report and one example task	Check the example repository licence	Results · Data
The harness tax	Public article with aggregate values	Not stated for request logs	Results